



南開大學
Nankai University

南 开 大 学

计 算 机 学 院

操作系统实验报告

Lab6: 进程调度

年级：2023 级

姓名：康志杰，王春晖，彭浩然

2026 年 1 月 1 日

目录

一、 实验目的	1
二、 实验原理	1
三、 实验过程	1
(一) 练习 0	1
(二) 练习 1	2
(三) 练习 2	4
1. 代码实现	4
2. 输出结果	6
3. 函数比较	7
4. Round Robin 调度算法的优缺点	8
5. 拓展思考	9
(四) challenge1	9
1. 代码实现	9
2. 实现结果	11
3. 多级反馈队列调度算法	11
4. 时间片数目和优先级成正比的原因	12
四、 实验总结	12
(一) 实验核心知识点与 OS 原理对应分析	12
(二) OS 原理中重要但实验未涉及的知识点	12

一、 实验目的

1. 理解操作系统的调度管理机制
2. 熟悉 ucore 的系统调度器框架，实现缺省的 Round-Robin 调度算法
3. 基于调度器框架实现一个 (Stride Scheduling) 调度算法来替换缺省的调度算法

二、 实验原理

操作系统的调度器负责在多个处于 PROC_RUNNABLE 状态的进程之间，决定“下一时刻由谁占用 CPU”。在 ucore 中，这一过程既可能由主动调度触发（如进程调用 sleep、wait、exit），也可能由被动调度触发（如时钟中断或系统调用返回前的调度检查）。无论触发方式如何，最终都会统一进入调度器框架，由调度器决定是否切换进程。

与实验五中将 FIFO 调度逻辑直接写入 schedule() 函数不同，实验六对调度系统进行了结构化改造：

kern/schedule/sched.c 中仅保留调度器框架，而将具体的调度策略封装为一个 sched_class 结构体。该结构体通过一组函数指针（init、enqueue、dequeue、pick_next、proc_tick）抽象出调度算法的行为，使得内核可以在不关心算法细节的前提下完成调度。

在此框架下，不同调度算法的本质区别，体现在运行队列的组织方式以及 pick_next 的选择策略上。

Round-Robin 通过链表维护就绪进程，强调时间公平；Stride 调度则通过优先队列维护最小 stride 值，强调资源分配与优先级成比例。这种“统一接口 + 不同数据结构”的设计，使调度算法具备良好的可扩展性。

三、 实验过程

（一） 练习 0

在实验之前需要将前面实验的代码进行补充优化。下面主要给出 lab6 更新的代码。分析 lab6 给出的代码可以观察到进程控制块 proc_struct 新增了属性 rq，rq 指向进程所属的运行队列，绑定进程与调度器的核心关联。run_link 是 rq->run_list 的链表节点 time_slice 是进程剩余 CPU 时间片。lab6_run_pool 是指向 rq->lab6_run_pool 的优先队列节点，在后面的 Stride 算法中会使用。lab6_stride 是当前进程的步长，lab6_priority 是进程优先级，这两个属性也是在 stride 算法中使用。

```
1 struct proc_struct
2 {
3     // ... 此处只给出新增代码片段
4     struct run_queue *rq;           // running queue contains Process
5     list_entry_t run_link;          // the entry linked in run queue
6     int time_slice;                 // time slice for occupying the
    CPU
7     skew_heap_entry_t lab6_run_pool; // FOR LAB6 ONLY: the entry in
    the run pool
8     uint32_t lab6_stride;            // FOR LAB6 ONLY: the current
    stride of the process
```

```

9      uint32_t lab6_priority;           // FOR LAB6 ONLY: the priority of
      process, set by lab6_set_priority(uint32_t)
10  };

```

在 proc.c 中 alloc_proc 函数实现进程控制块的初始化，所以要添加如下对于新增属性的初始化代码。将时间片，步长，优先级都初始化为 0，在之后会赋值。队列，队列节点都不用指向特定内容。

```

1  static struct proc_struct *alloc_proc(void)
2  {
3      struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
4      if (proc != NULL)
5      {
6          // ...
7          proc->rq = NULL;
8          memset(&proc->run_link, 0, sizeof(list_entry_t));
9          proc->time_slice = 0;
10         memset(&proc->lab6_run_pool, 0, sizeof(skew_heap_entry_t));
11         proc->lab6_stride = 0;
12         proc->lab6_priority = 0;
13     }
14     return proc;
15 }

```

此外还需要在 trap.c 中的时钟中断处理中添加这次实现的函数 sched_class_proc_tick 的调度。之后会说明在这里调用该函数的作用。其他补充代码的地方只需要将前面的代码进行复制。

```

1  case IRQ_S_TIMER:
2      clock_set_next_event();
3      ticks++;
4      // lab6: YOUR CODE (update LAB3 steps)
5      // 在时钟中断时调用调度器的 sched_class_proc_tick 函数
6      sched_class_proc_tick(current);
7
8      break;

```

(二) 练习 1

在实验六中，ucore 对调度系统进行了较为重要的结构性调整，将原本与具体调度策略紧密耦合的代码拆分为统一的调度器框架和独立的调度算法实现。这一改动使我在阅读代码时，能够更加清楚地区分“调度机制本身”和“调度策略差异”之间的关系，也直观体现了操作系统中常见的“机制与策略分离”设计思想。

调度器框架的核心抽象是 sched_class 结构体，其定义位于 kern/schedule/sched.h 中。该结构体通过一组函数指针，规定了一个调度算法在 ucore 中需要实现的基本接口，包括运行队列的初始化、进程的入队与出队、下一个运行进程的选择以及时钟中断到来时的处理方式。其定义形式如下：

```

1  struct sched_class {
2      const char *name;

```

```

3 void (*init)(struct run_queue *rq);
4 void (*enqueue)(struct run_queue *rq, struct proc_struct *proc);
5 void (*dequeue)(struct run_queue *rq, struct proc_struct *proc);
6 struct proc_struct *(*pick_next)(struct run_queue *rq);
7 void (*proc_tick)(struct run_queue *rq, struct proc_struct *proc);
8 };

```

从代码中可以看出, sched_class 并不包含任何具体调度逻辑, 而只是描述“调度算法应该具备哪些行为”。内核在调度过程中只通过这些接口与调度算法交互, 而并不关心算法内部是基于链表还是优先队列实现。这种通过函数指针实现的调用方式, 在 C 语言环境下起到了类似面向对象多态的效果, 使得不同调度算法可以被灵活替换。

与调度类相配合的是运行队列 run_queue 结构。在实验六中, 该结构相比实验五有了明显扩展, 不仅保留了用于 RR 调度的链表结构, 还新增了用于 Stride 调度的优先队列支持:

```

1 struct run_queue {
2     list_entry_t run_list;
3     unsigned int proc_num;
4     int max_time_slice;
5     skew_heap_entry_t *lab6_run_pool;
6 };

```

其中, run_list 用于维护 RR 调度下的就绪进程顺序, 而 lab6_run_pool 则作为斜堆的根指针, 支持 Stride 调度中对最小 stride 进程的高效选择。这一设计让我认识到, 调度框架并未假设所有调度算法都采用相同的数据结构, 而是根据算法需求进行扩展, 使框架本身具有更好的通用性。

从调度器的运行流程来看, 调度框架在系统启动早期即被初始化。在 init/init.c 中调用 sched_init(), 完成运行队列的初始化并绑定默认调度类。之后, 无论是新创建的进程, 还是从睡眠状态被唤醒的进程, 都会通过调度框架统一进入运行队列。例如, 在 wakeup_proc() 中, 进程被唤醒后并不会直接操作运行队列, 而是通过当前调度类的接口完成入队操作:

```

1 sched_class->enqueue(rq, proc);

```

这一实现方式使得进程状态的改变与具体调度算法的队列维护完全解耦, 调度框架本身无需关心入队顺序或数据结构细节。

真正触发进程切换的函数是 schedule(), 其实现位于 kern/schedule/sched.c 中。与实验五不同, 实验六中的 schedule() 不再直接包含任何调度策略, 而是通过调度类接口选择下一个运行进程, 其核心代码如下:

```

1 struct proc_struct *next = sched_class->pick_next(rq);

```

可以看到, schedule() 仅负责完成调度流程控制和上下文切换, 而“选谁运行”这一决策完全交由具体调度算法实现。这一改动是实验六能够同时支持 RR 与 Stride 调度的关键, 如果仍在 schedule() 中写死调度策略, 将无法实现算法的灵活切换。

在调度的触发机制上, 时钟中断起到了关键作用。每次时钟中断到来时, 内核都会调用当前调度类的 proc_tick 接口, 对正在运行进程的时间片进行更新。当进程的时间片耗尽时, 会在其进程控制块中设置 need_resched 标志, 例如在 RR 调度中可以看到类似如下逻辑:

```

1 if (proc->time_slice == 0) {
2     proc->need_resched = 1;
3 }

```

内核并不会在中断处理过程中立即进行进程切换，而是在中断返回前统一检查该标志，并在需要时调用 `schedule()`。这种通过标志位延迟触发调度的方式，使调度行为发生在安全、可控的时机，避免了在中断处理中直接进行上下文切换所带来的复杂性。

总体而言，通过对实验六调度器框架代码的阅读与分析，我对 `ucore` 中调度系统的整体设计有了较为清晰的认识。该框架通过统一接口屏蔽了调度算法差异，使内核主体逻辑保持稳定，同时又为 `RR`、`Stride` 等不同调度策略的实现提供了良好的扩展基础，这一设计思想与操作系统原理课程中强调的系统可扩展性高度一致。

(三) 练习 2

1. 代码实现

本次实验要实现 `RR` 调度算法。首先分析本次实验需要用到的数据结构，`sched_class` 是调度器结构体，包括调度器名称，以及调度器内实现的接口函数，`init` 函数初始化运行队列，`enqueue` 将进程加入运行队列，`dequeue` 将进程从运行队列移除。`pick_next` 从运行队列中选择下一个要执行的进程，`proc_tick` 进行时钟相应处理，时间片递减等。这种思想是策略与机制分离的思想，和前面实验中内存管理器实现思想一样。

```

1 struct sched_class
2 {
3     const char *name;
4     void (*init)(struct run_queue *rq);
5     void (*enqueue)(struct run_queue *rq, struct proc_struct *proc);
6     void (*dequeue)(struct run_queue *rq, struct proc_struct *proc);
7     struct proc_struct *(*pick_next)(struct run_queue *rq);
8     void (*proc_tick)(struct run_queue *rq, struct proc_struct *proc);
9 };

```

接着就是运行队列，`run_list` 是管理可运行进程的队列，`proc_num` 记录队列中进程数量，`max_time_slice` 是最大时间片，`lab6_run_pool` 是 `stride` 算法要用到的优先级堆。

```

1 struct run_queue
2 {
3     list_entry_t run_list;
4     unsigned int proc_num;
5     int max_time_slice;
6     // For LAB6 ONLY
7     skew_heap_entry_t *lab6_run_pool;
8 };

```

然后就开始实现 `RR` 算法，`RR_init` 实现了队列的初始化操作，主要调用函数 `list_init` 初始化 `run_list`，接着初始化队列进程数量为 0，因为初始化状态还没有进程加入队列。最大时间片在这里不需要初始化。

```

1 static void RR_init(struct run_queue *rq)
2 {
3     list_init(&(rq->run_list));
4     rq->proc_num=0;
5
6 }

```

RR_enqueue 函数实现了将进程插入队列的操作。首先判断队列和进程控制块是否为 NULL, 如果是直接返回不做任何处理。如果这个进程的剩余时间片为 0, 就是进程恰好用完时间片就被回收, 将该进程的时间片设置为最大时间片并将进程加入队列末尾。添加到队列末尾调用方法 list_add_before, 而不是 list_add_after。这是因为 rq->run_list 是双向队列, 其实这个节点是一个标志节点, 该节点前面一个节点就是双向队列的末尾, 通过这种方法插入更高效。最后把 rq 中的进程数目自增 1。

```

1 static void RR_enqueue(struct run_queue *rq, struct proc_struct *proc)
2 {
3     if (rq == NULL || proc == NULL) {
4         return;
5     }
6     if (proc->time_slice == 0)
7         proc->time_slice = rq->max_time_slice;
8     list_add_before(&rq->run_list, &proc->run_link);
9     rq->proc_num++;
10
11 }

```

RR_dequeue 实现的是将进程从队列中移除的操作, 这个函数实现很简单只需要调用函数 list_del_init 删除 proc->run_link 指向的节点, 最后将进程数目自减 1 就完成了删除操作。但是最开始还是要判断队列, 进程或者进程数是否为空, 如果为空不做处理直接返回。

```

1 static void RR_dequeue(struct run_queue *rq, struct proc_struct *proc)
2 {
3     if (rq == NULL || proc == NULL || rq->proc_num == 0) {
4         return;
5     }
6     list_del_init(&proc->run_link);
7     rq->proc_num--;
8 }

```

RR_pick_next 函数实现从队列中选择下一个要运行的进程。首先判断 proc_num 是否为 0, 如果为 0 表示队列为空直接返回 NULL 即可。否则需要调用 list_next 将队列最前面的进程取出, 但是此时得到是队列的一个节点, 可以通过 le2proc 函数获得对应的进程控制块, 这和前面获取 page 的原理一样。最后将进程控制块指针返回即可。

```

1 static struct proc_struct *RR_pick_next(struct run_queue *rq)
2 {
3     if (rq->proc_num == 0) {
4         return NULL;
5     }
6     list_entry_t *le = list_next(&rq->run_list);
7     struct proc_struct *proc = le2proc(le, run_link);
8     return proc;
9 }

```

RR_proc_tick 是在每一次时钟中断调用。首先判断当前队列是否为空, 进程是否为 NULL, 如果满足其中一个条件直接返回。如果不满足则将当前正在运行的进程的剩余时间片减一。如果在减一后剩余时间片为 0, 那么就在这个进程的 need_resched 标记为 1 表示进程需要调度。前面

在时钟中断调用函数 `sched_class_proc_tick(current)` 其实就是对该函数的接口调用, 进程为正在运行的进程 `current`。在时钟中断处理完之后内核判断进程是否需要调度的时候就会进行调度。

```
1 static void RR_proc_tick(struct run_queue *rq, struct proc_struct *proc)
2 {
3     if (rq == NULL || proc == NULL || proc == idleproc) {
4         return;
5     }
6     proc->time_slice--;
7     if (proc->time_slice <= 0) {
8         proc->need_resched = 1;
9     }
10 }
11 }
```

实现完这几个函数之后完成默认调度器 `default_sched_class` 的初始化, 调度器名称为 `RR_scheduler`, 接口函数实例就是上面实现的几个函数。之后在 `sched_init` 函数中将 `sched_class` 赋值为 `RR` 调度器之后就实现了 `RR` 算法调度。

```
1 struct sched_class default_sched_class = {
2     .name = "RR_scheduler",
3     .init = RR_init,
4     .enqueue = RR_enqueue,
5     .dequeue = RR_dequeue,
6     .pick_next = RR_pick_next,
7     .proc_tick = RR_proc_tick,
8 };
```

2. 输出结果

在 lab6 的命令行窗口输入命令 `make grade` 得到如图1所示结果, 结果显示得分为 50 分, 说明 `RR` 调度算法实现成功。



```
kzhj@kzhj-VMware-Virtual-Platform: ~/公共/labcode/lab6
kzhj@kzhj-VMware-Virtual-Platform:~/公共/labcode/lab6$ make grade
priority:                (3.6s)
-check result:           OK
-check output:           OK
Total Score: 50/50
kzhj@kzhj-VMware-Virtual-Platform:~/公共/labcode/lab6$
```

图 1: `RR` 算法测试结果

3. 函数比较

在 sched.c 中 schedule 函数发生了较大的变化。可以看到前面实验中使用的调度函数是通过遍历整个进程链表找到就绪状态的进程去调度。首先这种方法时间复杂度较高，最坏的情况下需要遍历整个进程链表。该函数找到第一个可运行进程就调度，若该进程长期可运行，其他进程就没有获取 cpu 资源的权力。

而且这种方法没办法利用其他的调度算法，比如 RR 算法，stride 算法等，可扩展性较差，效率较低。

```

1 void schedule(void)
2 {
3     bool intr_flag;
4     list_entry_t *le, *last;
5     struct proc_struct *next = NULL;
6     local_intr_save(intr_flag);
7     {
8         current->need_resched = 0;
9         last = (current == idleproc) ? &proc_list : &(current->list_link);
10        le = last;
11        do
12        {
13            if ((le = list_next(le)) != &proc_list)
14            {
15                next = le2proc(le, list_link);
16                if (next->state == PROC_RUNNABLE)
17                {
18                    break;
19                }
20            }
21        } while (le != last);
22        if (next == NULL || next->state != PROC_RUNNABLE)
23        {
24            next = idleproc;
25        }
26        next->runs++;
27        if (next != current)
28        {
29            proc_run(next);
30        }
31    }
32    local_intr_restore(intr_flag);
33 }

```

本次实验对该函数进行了改进，利用调度器对进程进行调度，前面分析中已知调度器的实现思想是策略和机制分离的思想，在该函数里使用调度器可以利用多种调度算法，可扩展性好。而且调度的效率大大提升。

```

1 void schedule(void)
2 {
3     bool intr_flag;

```

```
4     struct proc_struct *next;  
5     local_intr_save(intr_flag);  
6     {  
7         current->need_resched = 0;  
8         if (current->state == PROC_RUNNABLE)  
9         {  
10            sched_class_enqueue(current);  
11        }  
12        if ((next = sched_class_pick_next()) != NULL)  
13        {  
14            sched_class_dequeue(next);  
15        }  
16        if (next == NULL)  
17        {  
18            next = idleproc;  
19        }  
20        next->runs++;  
21        if (next != current)  
22        {  
23            proc_run(next);  
24        }  
25    }  
26    local_intr_restore(intr_flag);  
27 }
```

4. Round Robin 调度算法的优缺点

优点

RR 算法实现简单仅需维护运行队列，进程的入队出队，时间片计数，内存开销低。而且没有优先级依赖，无需复杂的优先级计算，比如 stride 算法还需要堆排序。短时间片可保证进程快速响应，避免单个进程霸占 CPU 导致卡顿。并且所有进程获得相等的 CPU 时间片，实现了公平性。

缺点

RR 算法不区分进程类型，对 CPU 密集型进程不友好，频繁切换导致计算中断，无法利用长时间段的 CPU 资源。无法为关键进程分配更多 CPU 时间片，不适用于有优先级需求的场景。而且时间片过小会带来频繁上下文切换带来的不必要开销。

通过调整时间片优化系统性能

系统不同时刻运行的进程数量不同。如果低负载时即可运行进程数小于 CPU 核心数，这时可以增加时间片大小，降低上下文切换开销。如果是高负载时，即可运行进程数远大于 CPU 核心数，这时缩小时间片，保证所有进程都能被调度。

而且不同的系统还可以选择不同时间片，比如交互式系统对实时性有较高要求，这时该系统应该选择较小的时间片，如果是计算型系统，频繁的切换导致无法利用较长 CPU 时间计算，此时可以选择较长时间片。

设置 need_resched 标志的原因

前面分析过程中已知每次时钟中断都会调用函数 RR_proc_tick，进行当前进程时间片自减 1，并在时间片减为 0 时设置标志 need_resched 为 1，表示需要调度。如果没有这个标志，当前

进程会一直占用 CPU 导致其他进程无法分配 CPU 资源。而且调用该函数时正处于中断过程中, 无法执行进程切换, 因此不能直接调用 `schedule()`, 只能设置标记。中断处理完成后再检查该标志调用 `schedule()`, 保证调度操作的安全性。

5. 拓展思考

优先级 RR 算法核心思想是: 按优先级划分队列, 同优先级内进程按 RR 调度, 高优先级队列优先被调度。所以管理进程的队列不能只有一个, 应该按照优先级个数设置对应个数的优先级队列。而且调度算法里需要先在优先级高的队列里执行 RR 算法, 只有高优先级队列为空时, 才可以轮到下一个优先级的队列。

实现代码过程中发现只有一个队列, 所以当前实现不支持多核调度。为了实现多核调度需要为每一个核心分配一个队列, 并且在每个核心里执行 RR 调度算法。每个核心的进程数量也不一样, 为了实现负载均衡, 可以将进程较多的核心的队列中的进程分给空闲核心的队列里进行调度。

(四) challenge1

1. 代码实现

本次实验挑战要实现 Stride 算法。下面给出该算法函数的实现过程。首先 `stride_init` 先要实现队列的初始化。此时优先队列为空, 所以将优先队列头部元素 `lab6_run_pool` 赋值为 `NULL`, 进程数量 `proc_num` 初始状态下为 0。

```
1 static void stride_init(struct run_queue *rq)
2 {
3     list_init(&(rq->run_list));
4     rq->lab6_run_pool = NULL;
5     rq->proc_num = 0;
6 }
```

在实现 `stride_enqueue` 之前先分析 `skew_heap_insert` 函数, 该函数实现优先队列的插入操作。第一个参数就是优先队列的头节点, 第二个参数是即将插入的节点, 第三个参数是函数 `proc_stride_comp_f`, 该函数用于比较步长 `lab6_stride`。整个优先队列维护最小堆, 根节点就是步长最小的节点。

`stride_enqueue` 函数首先将进程的优先队列节点 `proc->lab6_run_pool` 插入优先队列中, 接着更新进程的时间片为最大时间片。进程的队列节点指向队列 `rq`。最后队列的进程数自增一。这样就完成了进程控制块插入队列的操作。

```
1 static void stride_enqueue(struct run_queue *rq, struct proc_struct *proc)
2 {
3     rq->lab6_run_pool = skew_heap_insert(rq->lab6_run_pool, &(proc->
4         lab6_run_pool), proc_stride_comp_f);
5     proc->time_slice = rq->max_time_slice;
6     proc->rq = rq;
7     rq->proc_num++;
8 }
```

`stride_enqueue` 函数实现的是将节点从优先队列中删除, 调用函数 `skew_heap_remove` 实现删除操作, 删除之后该函数返回更新的队列, 将其重新赋值给 `rq->lab6_run_pool`。删除之后将进程数 `proc_num` 减 1。

```

1 static void stride_dequeue(struct run_queue *rq, struct proc_struct *proc)
2 {
3     rq->lab6_run_pool = skew_heap_remove(rq->lab6_run_pool, &(proc->
4         lab6_run_pool), proc_stride_comp_f);
5     rq->proc_num--;
6 }

```

stride 算法每次选择 pass 最小的进程，在前面分析过程中已知优先队列根据 lab6_stride 维护了最小堆，堆的根节点 rq->lab6_run_pool 就是步长最小的节点。然后通过函数 le2proc 将其转换为进程控制块指针。接着需要对该进程的 pass 进行更新，最后返回找到的进程即可。

```

1 static struct proc_struct *stride_pick_next(struct run_queue *rq)
2 {
3     skew_heap_entry_t *head = rq->lab6_run_pool;
4     if (head == NULL) {
5         return NULL;
6     }
7     struct proc_struct *next_proc = le2proc(head, lab6_run_pool);
8     next_proc->lab6_stride += BIG_STRIDE / next_proc->lab6_priority;
9     return next_proc;
10 }

```

stride_proc_tick 函数实现和 RR 算法一样，在时钟中断中调用，每一次时钟中断时进程时间片减 1，如果减完之后发现时间片用光了，就将标志 need_resched 赋值为 1，表示需要重新调度。

```

1 static void stride_proc_tick(struct run_queue *rq, struct proc_struct *proc)
2 {
3     /* LAB6 CHALLENGE 1: 2311656 */
4     proc->time_slice--;
5     if (proc->time_slice <= 0)
6     {
7         proc->need_resched = 1;
8     }
9
10 }

```

最后呢还是和 RR 算法一样构造 Stride 算法调度器实例。调度器名称为 stride_scheduler，函数接口都是上面实现的对应方法。

```

1 struct sched_class stride_sched_class = {
2     .name = "stride_scheduler",
3     .init = stride_init,
4     .enqueue = stride_enqueue,
5     .dequeue = stride_dequeue,
6     .pick_next = stride_pick_next,
7     .proc_tick = stride_proc_tick,
8 };

```

2. 实现结果

实现好 stride 算法之后, 在 sched_init 函数中将调度器实例化为 stride_sched_class。输入 make qemu 得到如图2所示结果, 观察到输出结果和指导书上期望的输出结果一致。

```
sched class: stride_scheduler
++ setup timer interrupts
kernel_execve: pid = 2, name = "priority".
set priority to 6
main: fork ok,now need to wait pids.
set priority to 5
set priority to 4
set priority to 3
set priority to 2
set priority to 1
child pid 7, acc 372000, time 2010
child pid 6, acc 324000, time 2020
child pid 5, acc 240000, time 2020
child pid 4, acc 200000, time 2030
child pid 3, acc 124000, time 2030
main: pid 3, acc 124000, time 2030
main: pid 4, acc 200000, time 2030
main: pid 5, acc 240000, time 2040
main: pid 6, acc 324000, time 2040
main: pid 0, acc 372000, time 2040
main: wait pids over
sched result: 1 2 2 3 3
all user-mode processes have quit.
init check memory pass.
kernel panic at kern/process/proc.c:531:
  initproc exit.

kzhj@kzhj-VMware-Virtual-Platform:~/公共/labcode/lab6$
```

图 2: Stride 算法测试结果

3. 多级反馈队列调度算法

多级反馈队列调度算法的核心是多级优先级队列 + 动态调整策略。设置 N 级队列, 队列优先级递减, 新进程入最高优先级队列按 RR 调度。时间片耗尽未完成则降级, 低优先级队列时间片更长。低优先级进程定期进入较高优先级队列, 避免很长一段时间内得不到运行权。高优先级队列优先调度, 如果为空时调度下一级队列, 下面给出设计思路:

数据结构

先对进程结构体 proc_struct 进行扩展, mlfq_level 是当前队列级别、last_sched_tick 是最近一次调度时间。接着对运行队列 rq 进行扩展, 实现多级队列, 每级维护独立队列, 为每一级队列配置梯度时间片。

函数实现

入队: 新进程插入最高优先级队列队尾, 初始化时间片为对应队列最大值。调度时从高优先级到低优先级遍历队列, 选首个非空队列的队首进程。如果进程时间片耗尽但未完成则降级, 标记调度标志。而且每隔固定时间检查低优先级进程, 若超过超过设置时间未调度则升级至上一级。最后递减当前进程时间片, 耗尽触发降级。

接着就要实现接口函数。mlfq_init: 初始化各级队列、时间片, 进程数初始化。mlfq_enqueue: 按进程优先级入队, 更新调度时间。mlfq_pick_next: 选择下一个调度的进程, 高优先级优先, 返回队首进程。mlfq_proc_tick: 时间片管理, 当时间片耗尽时降级。

4. 时间片数目和优先级成正比的原因

已知 $P.\text{pass} = \text{BigStride} / P.\text{priority}$, BigStride 是一个大常数, P.priority 是进程的优先级, P.pass 是进程的单步步长。通过上式可以知道进程单步步长和进程优先级成反比。而 Stride 算法的调度是每次选择 Stride 最小的进程。足够长时间后, 所有进程的累计步长增长总量近似相等。此时调度次数和 pass 成反比, pass 越小的进程, 调度次数越多。而 pass 越小代表进程优先级越高, 说明调度次数和进程优先级成正比, 调度次数直接反映的是时间片数目, 所以时间片数目和进程优先级成正比。

四、 实验总结

(一) 实验核心知识点与 OS 原理对应分析

通过本实验, 我对操作系统中处理器调度相关的核心概念形成了从“理论描述”到“代码实现”的完整理解。课堂上所讲的抢占式调度, 在实验中不再只是一个抽象定义, 而是通过时钟中断 $\rightarrow$ proc_tick $\rightarrow$ need_resched $\rightarrow$ schedule() 这一完整路径被具体体现出来。尤其是在 RR 调度算法中, 每一次时钟中断都会减少当前进程的时间片, 当时间片耗尽后设置 need_resched 标志, 从而在中断返回前触发调度。这一过程让我直观地理解到, 抢占式调度并不是“强行打断进程”, 而是通过受控的中断机制, 在内核态完成对调度时机的判断。

时间片机制是本实验中另一个理解明显加深的知识点。在理论中, 时间片通常被描述为“限制进程连续运行时间的手段”, 而在实验中可以清楚地看到, 它具体体现为 proc_struct 中的 time_slice 字段, 并且完全由调度器维护。通过 RR 调度算法的实现, 我理解到时间片不仅关系到公平性, 还直接影响系统的响应性: 时间片过大, 交互进程响应变慢; 时间片过小, 又会导致频繁上下文切换, 增加系统开销。这种取舍在实验中虽然没有定量调优, 但已经可以通过运行现象直观感受到。

在 Stride 调度算法的实现过程中, 我对“优先级调度”这一原理有了更本质的认识。与 RR 中“所有进程机会均等”不同, Stride 调度通过数学方式将“优先级”转化为 stride 增长速度, 使得进程在长期运行中获得的 CPU 时间与其优先级成正比。实验中 priority 测试程序的输出结果, 清楚地展示了不同优先级进程累计运行时间的差异, 这让我认识到: 优先级调度并不是简单的“插队”, 而是一种长期统计意义上的资源分配策略。

此外, 本实验中的调度器框架设计也加深了我对操作系统整体架构思想的理解。通过 sched_class 中的函数指针, ucore 将“调度机制”和“调度策略”进行了解耦, 使内核主体代码不依赖于具体算法实现。这种设计思想与 OS 原理中强调的“机制与策略分离”高度一致, 也让我意识到操作系统不仅追求功能正确, 更追求结构上的可扩展性与可维护性。

(二) OS 原理中重要但实验未涉及的知识点

尽管本实验已经涵盖了多种经典调度思想, 但仍有一些在操作系统原理中非常重要的内容未能在实验中直接体现。首先是多核处理器调度问题。当前 ucore 仅支持单核 CPU, 因此只维护一个全局运行队列, 而在真实系统中, 多核调度需要考虑每个 CPU 的本地运行队列、负载均衡以及任务迁移成本, 这些都会显著增加调度器设计的复杂度。

其次是实时调度算法，例如 EDF (Earliest Deadline First) 或 Rate Monotonic 调度。这类算法强调时间约束而非公平性，广泛应用于实时系统和嵌入式系统中，与本实验中的 RR 和 Stride 调度在设计目标上有明显区别，但实验中并未涉及。

此外，实验中也未深入讨论调度器与底层硬件结构的关系，例如缓存亲和性 (cache affinity)、NUMA 架构对调度决策的影响等。在现代操作系统中，调度算法往往需要结合硬件特性进行优化，而不仅仅是基于进程状态做出决策。

总体来看，本实验为理解操作系统调度机制提供了一个扎实的起点，但也让我意识到，真正的调度系统远比实验环境复杂，这些未涉及的内容为后续深入学习操作系统内核实现指明了方向。